Cedric Richter

Postdoc at Luxembourg University

Website: https://cedricrupb.github.io
E-Mail: cedric.richter@mailbox.org

Research Interests

My research focuses on tools and techniques at the intersection of **Machine Learning**, **Software Engineering**, and **Formal Methods**. To this end, I developed methods for finding bugs by learning from previous developer's mistakes, for adapting and combining software verification tools, and for cooperations between AI and formal verification. As part of my research, I have contributed to successful verification tools that are highly ranked in the annual software verification competition SV-COMP.

Education

2021 – 2025 **Carl von Ossietzky Universität Oldenburg**, Oldenburg
Ph.D. in computer science, Grade: *summa cum laude*.
Advisor: Heike Wehrheim
Topic: Neural Bug Detection
Disputation: 16.05.2025

2017 – 2019 **Paderborn University**, Paderborn
M.Sc in computer science, with distinction.

Topic: Neural Algorithm Selection for Software Verification
Date: 09 / 2019
Thesis Supervisor: Heike Wehrheim
Grade: 1,2

2014 - 2017 **Paderborn University**, Paderborn
B.Sc in computer science.

Topic: Evolutionary Strategies for Timed Game Automata
Date: 10 / 2017
Thesis Supervisor: Heike Wehrheim
Grade: 1,4

Positions and Experience

Oct 2025 – present **Luxembourg University**, Luxembourg
Interdisciplinary Centre for Security, Reliability and Trust (SnT)
Postdoc in the SerVal Group with Michail Papadakis

April 2021 – Sep 2025 **Carl von Ossietzky Universität Oldenburg**, Oldenburg
Research assistant in the Formal Methods Group lead by Heike Wehrheim

Oct 2019 – Mar 2021 **Paderborn University**, Paderborn
Research assistant in the Formal Methods Group lead by Heike Wehrheim

Oct 2016 - Sep 2019 **Paderborn University**, Paderborn
Student assistant in the Formal Methods Group lead by Heike Wehrheim

April 2015 – Sep 2016 **Paderborn University**, Paderborn
Teaching assistant for undergraduate lectures, Basic Programming

Peer-reviewed Conference and Journal Publications

ICSE'25 **Cooperative Software Verification via Dynamic Program Splitting**
Cedric Richter, Marek Chalupa, Marie-Christine Jakobs, and Heike Wehrheim

50th International Conference on Software Engineering

FMSD'25 **Construction of verifier combinations from off-the-shelf components**
Dirk Beyer, Sudeep Kanav, Tobias Kleinert, and **Cedric Richter**

Formal Methods in System Design (Journal)

FASE'24 **Can ChatGPT support software verification?**
Christian Janßen, **Cedric Richter**, and Heike Wehrheim

International Conference on Fundamental Approaches to Software Engineering

TACAS'24 **BubaaK-SpLit: Split what you cannot verify**
Marek Chalupa and **Cedric Richter**

30th International Conference on Tools and Algorithms for the Construction and Analysis of Systems

SCP'24 **Ranged Program Analysis on Path Ranges**
Jan Haltermann, Marie-Christine Jakobs, **Cedric Richter**, and Heike Wehrheim

Science of Computer Programming (Journal)

ASE'23 **How to Train Your Neural Bug Detector: Artificial vs Real Bugs**
Cedric Richter and Heike Wehrheim

38th International Conference on Automated Software Engineering

SEFM'23 **Robustness Testing of Software Verifiers**
Florian Dyck, **Cedric Richter**, and Heike Wehrheim

International Conference on Software Engineering and Formal Methods

SEFM'23 **Ranged Program Analysis via Instrumentation**
Jan Haltermann, Marie-Christine Jakobs, **Cedric Richter,** and Heike Wehrheim

International Conference on Software Engineering and Formal Methods

FASE'23 **Parallel Program Analysis via Range Splitting**
Jan Haltermann, Marie-Christine Jakobs, **Cedric Richter**, and Heike Wehrheim

26th International Conference on Fundamental Approaches to Software Engineering

ASE'22 **Are Neural Bug Detectors Comparable to Software Developers on Variable Misuse Bugs?**
Cedric Richter, Jan Haltermann, Marie-Christine Jakobs, Felix Pauck, Stefan Schott, and Heike Wehrheim

37th IEEE/ACM International Conference on Automated Software Engineering

ICST'22 **Learning Realistic Mutations: Bug Creation for Neural Bug Detectors**

Cedric Richter and Heike Wehrheim

15th IEEE Conference on Software Testing, Verification and Validation

MSR'22 **TSSB-3M: Mining single statement bugs at massive scale**
Cedric Richter and Heike Wehrheim

19th International Conference on Mining Software Repositories

FASE'22 **Construction of Verifier Combinations Based on Off-the-Shelf Verifiers**
Dirk Beyer, Sudeep Kanav, and **Cedric Richter**

25th International Conference on Fundamental Approaches to Software Engineering

FASE'21 **CoVeriTest with Adaptive Time Scheduling**
Marie-Christine Jakobs and **Cedric Richter**

24th International Conference on Fundamental Approaches to Software Engineering

ASE'20 **Attend and Represent:**
A Novel View on Algorithm Selection for Software Verification
Cedric Richter and Heike Wehrheim

35th IEEE/ACM International Conference on Automated Software Engineering

JASE'20 **Algorithm Selection for Software Validation Based on Graph Kernels**

Cedric Richter, Eyke Hüllermeier, Marie-Christine Jakobs, and Heike Wehrheim

Journal of Automated Software Engineering 2020

TACAS'19 **PeSCo: Predicting Sequential Combinations of Verifiers**
Cedric Richter and Heike Wehrheim

25th International Conference on Tools and Algorithms for the Construction and Analysis of Systems

Teaching Experience

Teaching Assistant

Teaching assistantships involve preparing and presenting exercises, coordination of student assistants, preparing and grading exams, and organizing office hours.

Winter 2023 **Basics of Theoretical Computer Science**
Bachelor course – Languages, Turing machines, Complexity

Summer 2023 **Formal Methods**
Bachelor course – Petri Nets, CCS, Timed Automata, Z

Winter 2022 **Basics of Theoretical Computer Science**
Bachelor course – Languages, Turing machines, Complexity

Summer 2022 **Model Checking**
Master course – Modeling, LTL, CTL, Bisimulation

Winter 2021 **Practical Software Verification**
Master course – Development of custom Verification Technique

Summer 2021 **Software Analysis**
Master course – Dataflow Analysis, Abstract Interpretation, Slicing

Winter 2020 **Programming**
Bachelor course – Programming in Python

Summer 2020 **Modeling Software with Formal Methods**
Bachelor course – Petri Nets, CCS, Timed Automata, Z

Winter 2019 **Software Analysis**
Master course – Dataflow Analysis, Abstract Interpretation, Slicing

Mentor

Mentorships involve organizing regular office hours and support in seminar theses and presentations.

Winter 2024 **Logical Methods in AI Verification seminar**
Master seminar

Winter 2023	**Logical Methods in AI Verification seminar** Master seminar
Winter 2020	**Advanced Topics in Software Analysis seminar** Master seminar
Sommer 2020	**Advanced Techniques in Software Analysis seminar** Master seminar
Winter 2019	**Software Testing seminar** Master seminar

Advising and Mentoring

Master thesis	Melchisedek Hengue Tuomo. *Residual Program Generation For Ranged Symbolic Execution. 2021*
Bachelor thesis	Christian Janßen. *Generating loop invariants for program verification with ChatGPT. 2023* See paper at FASE'24
	Darwin Kaufmann. *Evaluierung von ChatGPT im Kontext der Software Verifikation. 2023*
	Florian Dyck. *Benchmarkgenerierung für Verifikationswerkzeuge mittels semantikerhaltender Transformation. 2022* See paper at SEFM'23

Reviewing and Service

Program commitees	IEEE/ACM International Conference on Automated Software Engineering (ASE), Research Track, 2025
	ACM International Conference of Software Engineering (FSE), Tool Demonstration Track, 2025
	International Conference on Software Engineering (ICSE), ShadowPC, 2025
	IEEE Security Development Conference (SecDev), 2024

	Competition on Software Verification (SV-COMP). Jury member, 2018 – 2023
Journal reviewer	Formal Methods in System Design (FORM), 2023
Artifact Evaluation commitees	International Conference on Software Engineering (ICSE), 2024
	International Conference on integrated Formal Methods (iFM), 2023
	International Conference on Formal Methods in Software Engineering (FormaliSE), 2023
	International Conference on Computer Aided Verification (CAV), 2022
	International Conference on Fundamental Approaches to Software Engineering (FASE), 2022 International Conference on integrated Formal Methods (iFM), 2022
	ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2021
	European Conference on Object-Oriented Programming (ECOOP), 2020
	International Conference on Tests and Proofs (TAP), 2019

Research Statement

My research focuses on tools and techniques at the intersection of **Machine Learning**, **Software Engineering**, and **Formal Methods**. To this end, I developed methods for finding bugs by learning from previous developer's mistakes, for adapting and combining software verification tools, and for cooperations between AI and formal verification.

Software influences many aspects of our daily lives. We use software to communicate via our smartphones and it even influences the safety of our cars that we use to commute every day. Due to its widespread use, it is more important than ever that software is *correct* and *reliable*. Without correctness, even small mistakes in crucial systems can have enormous consequences on our daily lives. My research goal is to support software developers in the process of creating correct software through *scalable* and *effective* program analysis techniques that incorporate recent advances in AI and Formal Software Verification.
My contributions in this space have been published in **top-tier software engineering conferences (ICSE and ASE) receiving over 312 citations** and are deployed in successful software verification tools which are **highly ranked in the software verification competition SV-COMP**.

Cooperative and Distributed Software Verification. In the process of creating correct and reliable software systems, I believe that software verification should play a central role. Software verification has the goal of formally proving the correctness of software with respect to a formal specification. Therefore, a software verifier can provide strong formal guarantees for the correctness of the software system at hand.
While software verification has become a mature research area and many software verifiers exist, the problem itself of proving program correctness is far from being solved. One reason for this is the general undecidability of the problem. Therefore, existing software verification tools often have noteworthy limitations in their accuracy and scalability. My research contributes to overcoming these limitations through *cooperation* and *parallelization*.
Cooperation in software verification allows multiple verifiers to collaboratively solve a common verification problem. However, existing cooperative methods typically rely on proprietary exchange formats, limiting the integration of off-the-shelf software verifiers. In our ICSE'25 publication [1], we propose encoding the verification progress directly into the program, introducing a new scheme called "Cooperation via Splitting". In this approach, we probe a verifier with a verification task, splitting the task whenever the verifier fails. Progress is tracked by incrementally splitting the unverified parts of the program, thereby excluding parts that are already verified. At the end the unverified parts are then merged into a residual verification problem suitable for subsequent verifiers. Our format-independent method enables combining any software verifiers, leveraging their complementary strengths to solve more verification problems together.
Program splitting, and program decomposition in general, is also an interesting strategy for parallelizing the verification process. In our FASE'23 publication [2], we proposed to

generalize ranged symbolic execution to arbitrary software verifiers. Ranged symbolic execution decomposes the process of symbolic execution by dividing the program into multiple program ranges. Each range can then be independently analyzed. In our FASE'23 publication, we generalized this approach to run different software verifiers (or the same verifier) in parallel on different program ranges. We show that running the same verifier in parallel on multiple ranges can boost the verification performance.

In my **future research**, I am excited about advancing distributed and cooperative software verification. Many verification techniques are optimized for single core executions. This naturally limits the path or states that a verifier can process in each time frame. The emergence of large-scale computing however offers the opportunity to scale up the verification process considerably. My research goal is to distribute software verifiers over thousands of CPUs which then cooperate to verify increasingly complex software systems. To achieve this goal, I see significant potential in program *decomposition* and *abstraction.* Program decomposition can be a great way for distributed software verification. By decomposing the program, each part can be analyzed in parallel without requiring excessive communication. However, as we have seen in our FASE'23 publication, the success of the parallelized verification process depends on the way the program decomposition is performed. With a suboptimal decomposition, a verifier might fail to compute the correct level of abstraction to verify the task. Therefore, I believe that we need novel program decomposition techniques tailored for software verification.
A path towards distributed *and* cooperative software verification could be achieved through exchange of program abstractions. A common failure mode of software verifiers is that they do not find the correct level of abstraction for abstracting the behavior of loops or recursive function calls. To support each other, I believe that we need novel techniques for exchanging program abstractions. This has already been proven effective in abstract interpretation but largely under-explored in general software verification.

AI and Software Verification through Neural Specification Inference. Although software verifiers can play a significant role in achieving correct and reliable software, they still require a precise and complete formal specification to operate effectively. Without a formal specification, a software verifier can neither provide guarantees over the correctness of software nor find bugs. Unfortunately, recent studies [3] suggest that software developers often have problems in providing precise and complete specifications. As a result, some software bugs might be missed because the correct behavior of the software at hand is *under-specified*.

During my PhD, I investigated neural network-based models – called neural bug detectors – that are trained to detect bugs in functions. These neural bug detectors learn to infer the intended behavior of programs and find deviations from it. Neural bug detectors often utilize natural information of the program including variable names, comments or documentations for this purpose. As a result, they can detect a variety of *semantic bugs* which traditional methods cannot without an explicit specification of program correctness.
Although neural bug detectors cannot provide guarantees over the correctness of their output, my research shows that there is a significant potential for *cooperation between AI and formal methods*. Neural models can infer a specification of program correctness from implicit information provided in the program. Formal software verification can formally prove whether an inferred specification holds or not. Even though the inferred specifications

are not necessarily correct, I believe that formally proving them can still be helpful for assessing the correctness of a system.

For example, when a software developer is confronted with violated specifications, the developer can decide which specification should be enforced (and hence require modifications of the system). Deciding whether a specification should hold or whether a violation represents incorrect behavior might be simpler than creating the specification at first. However, to evaluate this idea, we need AI systems that can generate formal specification which can be processed by software verifiers.
The emergence of increasingly powerful AI systems, in particular large language models (LLM), paves the way to achieving this idea in practice. As we show in our FASE'24 publication [4], LLMs can be effectively used to generate invariants in a structured format supported by existing software verifiers. By processing the LLM generated invariants, the verifiers were able to verify novel verification task that they could not solve before. Overall, my research shows that AI can be used for inferring specification and that AI can be used to generate invariants in a structure formatted.

In my **future research**, I am excited about combining the capabilities of modern AI systems and formal software verifiers. Ultimately, I envision a system that formally prove the correctness of software with *minimal* human intervention. With this system, the user does not have to provide a precise and complete formal specification. Instead, the user can focus on implementing the system while only providing minimal feedback on the inferred invariants and their violations. I believe that this system can be achieved with progress on *distributed and cooperative software verification,* which would lead to more effective and scalable software verifiers, and with progress on *cooperation between AI and formal methods.*

Cooperation and Projects. To achieve my research goals, I plan to continue and expand my research collaborations with national and internation researchers. My research on cooperative and distributed software verification is a joint effort between University of Oldenburg (Heike Wehrheim), LMU Munich (Marie-Christine Jakobs and Dirk Beyer) and the Institute of Science and Technology Austria (ISTA; Marek Chalupa). For my research on neural specification inference, we are currently in the process of submitting a DFG research proposal.

References

[1] Richter, C., Chalupa, M., Jakobs, M. C., & Wehrheim, H. (2025, March). Cooperative Software Verification via Dynamic Program Splitting. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (pp. 629-629). IEEE Computer Society.

[2] Haltermann, J., Jakobs, M. C., Richter, C., & Wehrheim, H. (2023, April). Parallel program analysis via range splitting. In *International Conference on Fundamental Approaches to Software Engineering* (pp. 195-219). Cham: Springer Nature Switzerland.

[3] Greenman, B., Saarinen, S., Nelson, T., & Krishnamurthi, S. (2022). Little tricky logic: misconceptions in the understanding of LTL. *arXiv preprint arXiv:2211.01677*.

[4] Janßen, C., Richter, C., & Wehrheim, H. (2024, April). Can ChatGPT support software verification?. In *International Conference on Fundamental Approaches to Software Engineering* (pp. 266-279). Cham: Springer Nature Switzerland.

Teaching Statement

Teaching and guiding students on their way to becoming successful computer scientists is one of my main motivations to pursue an academic career. I enjoy teaching students **fundamental concepts** of computer science and helping them turn what they have learned into **practical skills**.

Teaching and Mentoring Experience. During my career, I had many opportunities to refine and improve my teaching skills. I have served as a **teaching assistant (TA) for 9 classes:** I assisted with three introductory computer science classes (with up to 500 participants), two bachelor courses on formal methods and four master courses on model checking and software analysis. In addition, I organized and co-organized five master seminars on software testing, software analysis, and formal methods for verifying AI systems. As a TA, I have experience with preparing and presenting exercises, coordinating student assistants (for larger introductory courses), and with preparing and grading exams. For the seminars, I organized regular office hours where I supported the students in preparing and presenting their seminar works.

I am convinced that teaching can be improved through practice and with the help of evidence-based methods. That is why I participated in the voluntary program for qualification in university didactics. This program teaches effective pedagogy for university teachers and provided me with effective, research-based methods to improve my teaching skills. As part of the program, I attended 6 three-to-four-day workshops involving topics like engaging students in active learning, planning classes, designing exams, and techniques for providing and receiving constructive feedback from students.

I had the pleasure of mentoring bachelor and master students during the preparation of their final theses. I supervised the research of three bachelor students and one master student by proposing research ideas, organizing regular research meeting, and by providing feedback. Under my supervision, the students not only completed their thesis successfully, but they were also able to contribute to two research publications.

Teaching philosophy. As a teacher, my primary goal is to help students develop *practical skills* that will serve them throughout their careers and further education. Rather than a way to teach students facts to memorize, I believe that teaching and learning should be:

Active and interactive. Students should play an active role in their learning process. When students attend my classes, I want them to engage actively with me and the content of the lecture. My approach to achieving this is a combination of presentations and interactive elements. For example, in my exercises, I always try to develop the solution to a given problem together with the students. Instead of providing an example solution, I motivate students to present their own solution guiding them on the way to a correct solution. In larger lectures, I believe that it is important for the same reason to integrate interactive elements such as voting, multiple-choice questions or online tools for real-time feedback. This allows students to get direct feedback and reflect on their own understanding of the given topic.

Problem-driven and research-oriented. Computer scientists often face *open-ended* problems that require a combination of technical knowledge and creative thinking to solve. My goal is for students to leave my course equipped with practical tools and skills to solve *real-world challenges*. To achieve this, I use ideas from problem-driven teaching and research-oriented learning.

For example, in Winter 2021, I contributed to designing the "Practical Software Verification" course. The course initially introduced students to software verification fundamentals through lectures and exercises. The introductory phase concluded with a research idea for a novel cooperative verification scheme. In the practical phase, students then implemented and evaluated this idea in small groups. At the end, we organized a small competition where students researched how to apply the novel technique and competed to achieve the best verification results on a given benchmark.

Working on such open-ended research problems helps students develop critical thinking and problem-solving skills essential for both academia and industry.

Transparent and goal directed. While I have my teaching goals, students also have individual learning objectives, which may not always align. Thus, clearly communicating lecture and exercise goals and expectations is crucial. I often start exercises by presenting specific goals and conclude with reflections on their achievement. This reflection includes a close feedback loop, allowing both teachers and students to exchange feedback to enhance learning. For instance, incorporating short feedback sessions after each exercise has enabled me to promptly address student issues, such as organizing additional sessions to help students write mathematical proofs based on their feedback.

Teaching interests. My broad research interests and prior teaching experience have prepared me to teach bachelor and master courses on **Software Engineering, Software Analysis, Model Checking, Theoretical Computer Science,** as well as a variety of introductory computer science courses. I would be excited to teach existing classes in this area as well as to develop new classes, e.g.:

AI for Software Engineering: With the rise of powerful software engineering agents (e.g., GitHub Copilot or Devin AI), AI is becoming central to modern software engineering. However, we must critically examine this trend and its implications for software engineering practices. In this master's seminar, students can explore recent literature at the intersection of AI and software engineering, reflecting critically on developments. Given the topic's recency, extending this seminar into a project-focused seminar where students investigate novel applications is also feasible.

Formal Verification of AI systems. A course for advanced bachelor students and master students that explores recent advances in AI verification. The course provides a basic introduction in modern neural network-based AI systems (basic MLPs and CNNs). Then it introduces recent model checking and abstract interpretation techniques that are utilized for AI verification. As the course intersects with topics of the KI and Data Science department, I could imagine developing this course in collaboration.